Funciones en C: entradas, trabajo y resultado

Guía para explicar en clase y consultar durante las prácticas.
IFCT0609 · MF0964_3 · Unidad formativa UF1286

Una función es un bloque de instrucciones con un nombre que realiza una tarea. Puede recibir datos, trabajar con ellos y devolver un resultado. Al dividir un programa en funciones, cada parte tiene una responsabilidad más fácil de comprender, probar y reutilizar.

La duda que vamos a resolver

Si escribimos int main(void), ¿solo podemos utilizar variables enteras? No. El primer int indica el tipo del resultado que devuelve la función. Dentro de ella podemos declarar variables de otros tipos y utilizar funciones que muestran texto o consultan datos.

Tres preguntas diferentes	Ejemplo
¿Qué datos recibe?	Los parámetros que aparecen entre paréntesis.
¿Con qué datos trabaja dentro?	Las variables que declara o utiliza en su cuerpo.
¿Qué resultado devuelve?	La expresión que acompaña a return y el tipo declarado antes del nombre.

Objetivos de esta guía

Distinguir parámetros, variables internas y retorno; leer int main(void); llamar a una función con dos argumentos; seguir el recorrido de los datos; diferenciar mostrar y devolver; comprender la forma básica de main con argumentos de la línea de órdenes.

Los ejemplos están pensados para C11 en un entorno habitual de consola. Al final se incluyen ejercicios y orientaciones para el profesor. Las tres infografías complementarias resumen las ideas para proyectarlas.

Cómo se lee int main(void)

```
int main(void)
{
    /* Aquí escribimos las instrucciones. */
    return 0;
}
```

Elemento	Significado
int	El tipo del valor que devuelve main es entero. No determina el tipo de todas sus variables.
main	Nombre especial de la función principal en un programa habitual de consola.
(void)	La función no recibe parámetros. No significa que no pueda pedir datos al usuario.
{ ... }	Delimitan el cuerpo: las instrucciones de la función.
return 0;	Termina main y comunica una finalización correcta al entorno que ejecuta el programa.

Cuando se lanza el ejecutable, el entorno de ejecución prepara lo necesario y llama a main. Desde main se ejecutan las instrucciones y se llama a otras funciones. Por eso la presentamos al alumnado como la entrada a su código, aunque antes exista trabajo de preparación del entorno.

Qué representa el cero

En main, devolver 0 indica éxito. EXIT_SUCCESS, declarado en stdlib.h, también expresa éxito. EXIT_FAILURE expresa fallo de forma portable. Otros valores distintos de cero suelen utilizarse para errores, pero su interpretación depende del entorno.

En C11, llegar a la llave final de main equivale a devolver 0. En estos ejemplos escribimos return 0 explícitamente para que el cierre del programa resulte visible.

Variables internas y valor devuelto

```
#include <stdio.h>

int main(void)
{
    char inicial = 'A';
    int edad = 25;
    double altura = 1.75;

    printf("Inicial: %c\n", inicial);
    printf("Edad: %d\n", edad);
    printf("Altura: %.2f\n", altura);
    return 0;
}
```

La función utiliza un carácter, un entero y un número decimal. Las llamadas a printf muestran esos datos. Cuando se alcanza return 0, el valor devuelto por main es el entero 0. No se devuelve ni la inicial, ni la edad, ni la altura.

Dato o instrucción	Papel
char inicial = 'A';	Variable interna que guarda un carácter.
double altura = 1.75;	Variable interna que guarda un número decimal.
printf(...)	Muestra información en la salida estándar, normalmente la pantalla.
return 0;	Devuelve el resultado de main al entorno de ejecución.

Carácter y cadena no son lo mismo: 'A' es un carácter; "Ana" es una cadena. Podemos guardar una cadena con char nombre[] = "Ana"; y mostrarla mediante printf("%s", nombre);. Esto tampoco cambia el tipo de retorno de main.

Idea para repetir en clase: el tipo que aparece antes del nombre indica qué devuelve la función; no limita los tipos con los que trabaja en su interior.

Mostrar en pantalla no es devolver

```
#include <stdio.h>

int obtener_edad(void)
{
    printf("Estoy consultando la edad\n");
    return 25;
}

int main(void)
{
    int edad = obtener_edad();
    printf("Edad recibida: %d\n", edad);
    return 0;
}
```

Al llamar a obtener_edad(), primero se muestra un mensaje. Después, return 25 entrega un entero a main. Ese entero se guarda en edad. Finalmente main muestra la edad y devuelve su propio código de finalización: 0.

Operación	Destinatario y finalidad
printf("mensaje");	Escribe información en la salida estándar para que pueda verla el usuario o recogerla otro programa.
return 25;	Entrega un valor al código que llamó a obtener_edad y termina esa llamada.
return 0; en main	Comunica al entorno el resultado de finalización del programa.

printf también devuelve un entero: la cantidad de caracteres escritos, o un valor negativo si se produce un error. En estos ejemplos no usamos su valor de retorno. Su efecto visible es escribir texto; son dos aspectos distintos de una misma llamada.

Un ejemplo deliberadamente sencillo

obtener_edad devuelve aquí un 25 fijo para observar el mecanismo. En una aplicación real podría obtenerlo de un cálculo o de datos previamente validados. Lo relevante es seguir adónde va el valor devuelto.

Dos parámetros: recibir datos y calcular

```
#include <stdio.h>

int sumar(int numero1, int numero2)
{
    return numero1 + numero2;
}

int main(void)
{
    int resultado = sumar(4, 6);
    printf("Resultado: %d\n", resultado);
    return 0;
}
```

Expresión	Interpretación
int sumar(...)	La función sumar devuelve un entero.
int numero1, int numero2	Declara dos parámetros de tipo entero.
sumar(4, 6)	Llama a sumar con los argumentos 4 y 6.
return numero1 + numero2;	Calcula la suma y entrega su valor a quien llamó.
int resultado = sumar(4, 6);	Guarda el valor devuelto en una variable de main.

Parámetro es el nombre que aparece en la definición, como numero1. Argumento es el valor o expresión que se proporciona al llamar, como 4. Pueden tener tipos diferentes: una función puede recibir char e int y devolver int.

En este ejemplo usamos números pequeños para concentrarnos en la llamada. El tipo int tiene un rango limitado; una suma que lo desborde no es válida para este diseño y exige comprobaciones adicionales.

Recorrido de una llamada y paso por valor

Paso	Qué ocurre
1. main llama	Se evalúa sumar(4, 6). La asignación queda pendiente hasta obtener el resultado.
2. Se reciben los valores	numero1 toma 4 y numero2 toma 6. Son parámetros locales de esa llamada.
3. Se calcula	numero1 + numero2 produce 10.
4. Se devuelve	return 10 termina sumar y entrega el valor a main.
5. Se continúa	resultado recibe 10 y main ejecuta la siguiente instrucción.

Los enteros se pasan por valor

```
#include <stdio.h>

int aumentar(int numero, int incremento)
{
    numero = numero + incremento;
    return numero;
}

int main(void)
{
    int original = 5;
    int nuevo = aumentar(original, 2);
    printf("Original: %d\n", original);
    printf("Nuevo: %d\n", nuevo);
    return 0;
}
```

El resultado es Original: 5 y Nuevo: 7. numero recibe una copia del valor de original. Modificar numero no modifica original. Para conservar el resultado, lo guardamos en nuevo. Si escribiéramos original = aumentar(original, 2);, original pasaría a valer 7 por la asignación en main.

C pasa los argumentos por valor. Más adelante, al utilizar punteros, podremos pasar una dirección y modificar el objeto al que apunta. El propio valor de la dirección también se copia al parámetro.

Recibir un carácter y devolver un entero

```
#include <stdio.h>

int comprobar_letra(char letra, int edad)
{
    if (letra == 'A' && edad >= 18)
    {
        return 1;
    }
    return 0;
}

int main(void)
{
    int cumple = comprobar_letra('A', 20);
    printf("Cumple la condicion: %d\n", cumple);
    return 0;
}
```

La función recibe dos datos distintos: un carácter y una edad entera. Devuelve un entero que representa una decisión: 1 si se cumplen las dos condiciones y 0 en caso contrario. Estos valores son el contrato que hemos elegido para esta función.

Argumentos	Resultado
'A', 20	1: se cumplen las dos condiciones.
'B', 20	0: la letra no coincide.
'A', 17	0: la edad es menor que 18.
'A', 18	1: el límite 18 está incluido.

No confundir los convenios: comprobar_letra usa 1 para una condición verdadera. main usa 0 para comunicar éxito al entorno. El significado del número depende de la función y debe explicarse. Este ejemplo supone una edad válida, ya comprobada por quien llama.

void, declaraciones y definición de funciones

Cabecera	Qué indica
int obtener_edad(void)	No recibe parámetros y devuelve un entero.
int sumar(int a, int b)	Recibe dos enteros y devuelve un entero.
void mostrar_aviso(void)	No recibe parámetros y no devuelve un valor.
void mostrar_numero(int numero)	Recibe un entero y no devuelve un valor.

```
void mostrar_aviso(void)
{
    printf("Atencion: revise los datos\n");
    return;
}
```

Este fragmento supone que se ha incluido stdio.h. En una función void, return; permite salir sin entregar un valor. También podemos llegar a su llave final. La palabra void tiene aquí dos posiciones: antes del nombre habla del retorno; entre paréntesis habla de la ausencia de parámetros.

¿Dónde escribimos cada función?

En los programas anteriores definimos las funciones auxiliares antes de main. Si queremos definirlas después, debemos declararlas antes de su primera llamada mediante un prototipo:

```
int sumar(int a, int b);  /* Declaracion: termina en ; */

int main(void)
{
    int total = sumar(4, 6);
    return total == 10 ? 0 : 1;
}

int sumar(int a, int b)   /* Definicion: incluye cuerpo */
{
    return a + b;
}
```

La declaración comunica al compilador el nombre, el retorno y los parámetros. La definición aporta las instrucciones. En C11, escribimos (void) para declarar explícitamente que no hay parámetros; evitamos la forma vacía () al enseñarlo.

Los dos parámetros especiales de main

En un programa habitual de consola, main puede recibir los argumentos de la línea de órdenes con esta forma estándar:

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    printf("Cantidad de argumentos: %d\n", argc);
    for (int i = 0; i < argc; i++)
    {
        printf("argv[%d] = %s\n", i, argv[i]);
    }
    return 0;
}
```

Parámetro	Significado
int argc	Cantidad de argumentos. En una ejecución habitual incluye el nombre del programa.
char *argv[]	Permite acceder a los argumentos como cadenas de caracteres. La sintaxis de punteros se estudiará después.

```
./argumentos Ana 20

Cantidad de argumentos: 3
argv[0] = ./argumentos
argv[1] = Ana
argv[2] = 20
```

El nombre mostrado en argv[0] depende de cómo se lance el programa. Si argc es 3, los textos se consultan en los índices 0, 1 y 2; argv[argc] es un puntero nulo. Antes de utilizar argv[1] o argv[2] hay que comprobar que existen.

"20" llega como texto, no como int. Para calcular con él tendríamos que convertirlo y validar la conversión y el rango, por ejemplo con strtol. No basta con cambiar el nombre o el tipo de una variable.

No escribimos int main(int a, int b) para sumar dos números. Para eso creamos una función propia. También podemos mantener main(void) y leer datos del teclado durante la ejecución: parámetros y entrada de teclado son mecanismos diferentes.

Ejercicios para trabajar en clase

Antes de ejecutar, el alumnado debe escribir su predicción. Después compara lo esperado con lo observado y explica cualquier diferencia.

1. Identificar las tres partes

```
int evaluar(char letra, int edad)
{
    double limite = 18.0;
    return letra == 'A' && edad >= limite;
}
```

Indica el tipo devuelto, los parámetros y sus tipos, y la variable interna. ¿Es válido utilizar double dentro de esta función que devuelve int? Justifica la respuesta.

2. Trazar una llamada

Con sumar definida como en la página 5, completa para int r = sumar(7, 3): valor de numero1, valor de numero2, valor devuelto y valor final de r. ¿Se imprime algo si no hay una llamada a printf?

3. Escribir una función con dos parámetros

Crea int mayor(int a, int b), que devuelva el mayor de dos enteros. Prueba (4, 9), (9, 4), (5, 5) y (-3, -8). Muestra el resultado desde main. Si son iguales, devuelve ese mismo valor.

4. Diferenciar mostrar y devolver

Explica por qué printf("Hola"); y return 0; no tienen la misma función. Después, crea void mostrar_inicial(char inicial), que muestre un carácter sin devolver un valor.

5. Observar el paso por valor

En el ejemplo aumentar, cambia la llamada por aumentar(original, 2); sin guardar su resultado. ¿Cuánto vale original después? ¿Qué información se pierde? Propón cómo conservar el nuevo valor.

6. Comprender argc y argv

Ejecuta el ejemplo de argumentos como ./argumentos Ana 20 y como ./argumentos "Ana Maria" 20. Cuenta los argumentos de cada ejecución y explica el efecto de las comillas. No accedas a posiciones que no existan.

Ampliación

Crea int es_mayor_de_edad(int edad), que devuelva 1 desde los 18 años y 0 antes. Su entrada debe ser una edad ya validada. Prueba 17, 18 y 19 y explica por qué devolver 0 aquí no representa un error del programa.

Orientaciones y respuestas para el profesor

Esta página contiene soluciones. Para una entrega de ejercicios sin respuestas, imprimir únicamente las páginas 1 a 10. Las infografías se facilitan en un PDF aparte.

Respuestas esperadas

1. Retorno int; parámetros letra de tipo char y edad de tipo int; variable interna limite de tipo double. Es válido: las variables internas no están limitadas por el tipo de retorno. Las comparaciones y && producen un entero 0 o 1.

2. numero1 = 7, numero2 = 3, retorno = 10, r = 10. No se imprime nada por devolver un valor. 3. Los resultados son 9, 9, 5 y -3:

```
int mayor(int a, int b)
{
    if (a >= b)
    {
        return a;
    }
    return b;
}
```

4. printf escribe; return termina la llamada y puede entregar un valor. La función solicitada puede escribirse así, con stdio.h incluido:

```
void mostrar_inicial(char inicial)
{
    printf("%c\n", inicial);
}
```

5. original sigue valiendo 5. Se descarta el 7 devuelto. Guardarlo con int nuevo = aumentar(original, 2); o asignarlo a original conserva el resultado.

6. Ambas ejecuciones habituales tienen argc = 3. En la segunda, argv[1] contiene Ana Maria como un solo argumento por las comillas del intérprete de órdenes. En la primera contiene Ana. argv[2] contiene "20" en ambos casos.

Ampliación: return edad >= 18; devuelve 0, 1 y 1 para las tres pruebas. El 0 indica que la condición es falsa; no es el estado de salida de main.

Secuencia de explicación sugerida: 60-75 minutos

10 min: leer main y separar los tres tipos de información. 10 min: ejecutar variables y mostrar/devolver. 15 min: seguir sumar y aumentar con una traza en la pizarra. 10 min: explicar parámetros mixtos y void. 15-30 min: ejercicios, main con argumentos y puesta en común. Ajustar el ritmo al grupo.

Compilar, ejecutar y comprobar

Los seis programas completos se entregan también en un ZIP. Cada archivo contiene su propio main: se compilan por separado. No se deben unir todos en un único ejecutable.

```
gcc -std=c11 -Wall -Wextra -pedantic 03_sumar.c -o sumar

# Linux / terminal de entorno compatible
./sumar

# Windows PowerShell, con GCC instalado
.\sumar.exe
```

En Windows puede compilarse con -o sumar.exe. El compilador debe estar instalado y accesible desde el terminal. VS Code es el editor; una extensión no sustituye al compilador. En WSL se siguen las órdenes de Linux dentro del terminal de WSL.

Archivo	Qué observar
01_variables.c	Tipos internos diferentes; main devuelve 0.
02_mostrar_devolver.c	El mensaje y el retorno 25 son cosas distintas.
03_sumar.c	Dos argumentos producen un resultado 10.
04_por_valor.c	original conserva 5; nuevo recibe 7.
05_comprobar.c	Parámetros char e int; resultado 1 o 0.
06_argumentos_main.c	argc y argv dependen de la orden de ejecución.

Errores que conviene corregir en la pizarra

Confundir printf con return; pensar que int obliga a usar solo enteros; escribir "A" donde se quiere el carácter 'A'; usar void y return 5 en la misma función; llamar a sumar con una cantidad incorrecta de argumentos; escribir funciones sin declaración previa; tratar argv[2] como número sin conversión; acceder a un argumento inexistente.

Comprobación final de comprensión

Pedir a cada pareja que señale en un ejemplo: qué entra, dónde se calcula, qué se muestra y qué se devuelve. La explicación debe distinguir el resultado de una función auxiliar del estado de salida que devuelve main.

Frase de cierre para el alumnado: una función puede recibir unos tipos de datos, utilizar otros en su interior y devolver el tipo que declara en su cabecera.